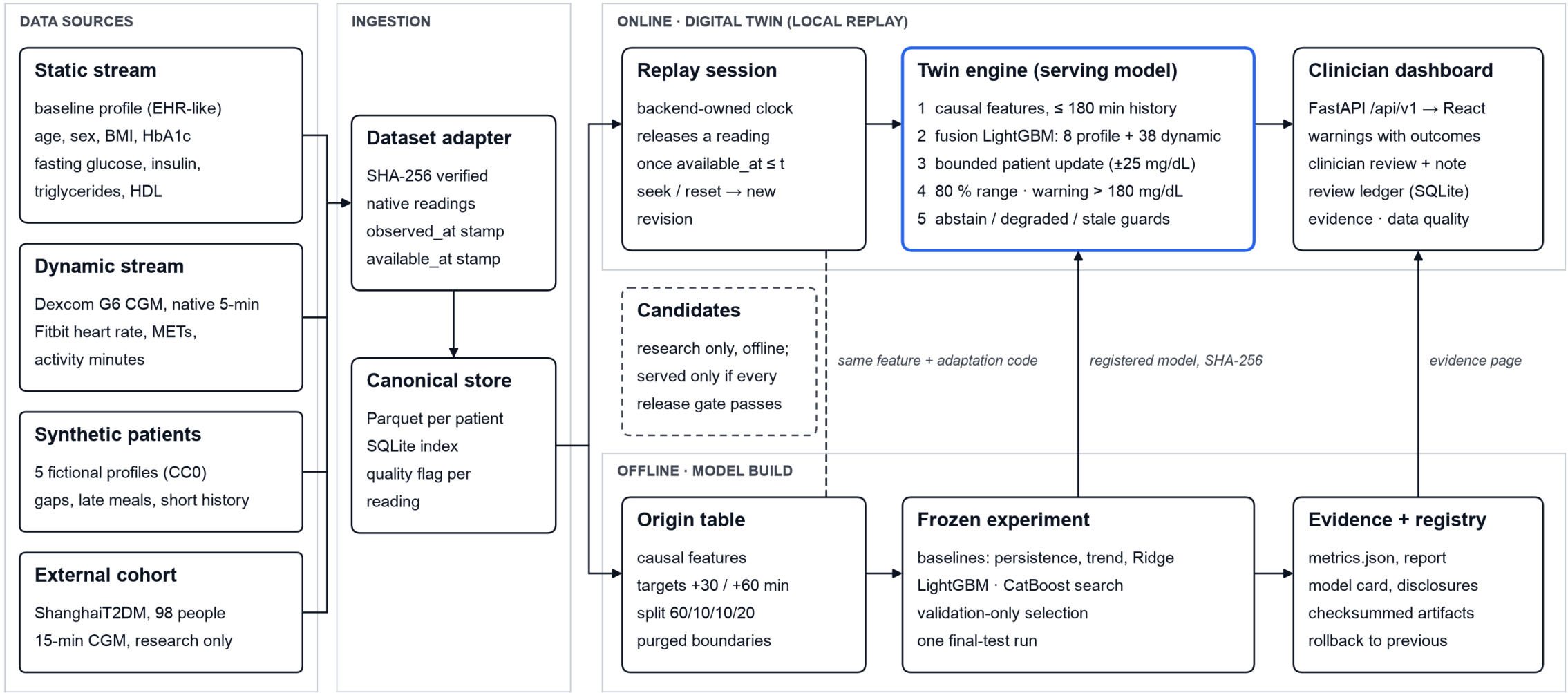
GlucoTwin — system architecture

Two data streams fused in one model; the offline build and the online twin share the same feature and adaptation code.



Solid arrow: data flow · Dashed line: identical code offline and online · Blue: serving model · Dashed box: research only, not served · CPU only, local, no cloud, no LLM · not a medical device

Forecast lifecycle, availability boundary and state ownership

1. Origin. A native glucose reading becomes available at time t. Features use only records with observed_at ≤ t and available_at ≤ t (glucotwin.features.build_features — the same code offline and online).

2. Population model. LightGBM predicts glucose change at +30 and +60 minutes from 46 features in one model (feature set dyn-prof-v2): 38 trailing glucose and wearable features (each 5-minute wearable summary counts only the part inside the lookback window) and 8 static baseline profile values (age, sex, BMI, HbA1c, fasting glucose and insulin, triglycerides, HDL), assumed known at the start of the record; the model abstains when any of them is unavailable. Artifacts carry a transformation fingerprint; mismatched weights are refused.

3. Bounded patient update. AdaptationState adds a shrunk, decaying correction learned only from earlier forecasts whose target readings were already available: at least 6 matured outcomes, capped at ±25 mg/dL, reset after a 3-hour gap. Model weights are never refit.

4. Uncertainty and warning. Split-conformal prediction ranges (target coverage 80 %) calibrated for this exact pipeline; warning score = max forecast within 60 min – 180 mg/dL (uncalibrated, not a probability). Warning outcomes use one shared one-to-one onset matcher offline and in replay.

5. Immutable record and review ledger. Forecasts are stored once under uuid5(session, revision, origin, model version); outcomes are revealed only when the replay clock passes their availability. A clinician's review of a warning (with a note) is written to the SQLite audit; the review summary freezes what was known at its replay cutoff.

Candidates. Research experiments write only their own artifacts and never the model registry; a candidate is served only after every declared release gate, serving parity and integration checks pass, through an explicit promote command with a recorded rollback.

6. Revisions. Seek/reset rebuild the twin from released events only; old cursors return 409; the browser keys caches by (session, revision, clock).

State ownership. Canonical store (Parquet + SQLite, local) · replay clock and sessions (API memory) · twin state (per session revision, rebuilt deterministically) · issued forecasts and acknowledgements (SQLite audit) · model artifacts (checksummed registry) · UI preferences (browser).

Posture. CPU only; loopback binding; narrow CORS; no cloud, API keys or LLMs; trusted artifacts verified by SHA-256; no pickles. Research demonstration, not production or a medical device.